



BASE DE DATOS AVANZADAS Y BIG DATA

UPN.EDU.PE

Semana 03



Funciones Almacenadas.
Cursores.



Presentación de la sesión

Logro de sesión

Al final de la sesión el alumno podrá crear cursores, y funciones almacenadas para la aplicación de la lógica de programación en la base de datos.

Temario

- Cursores.
- Funciones almacenadas



¿Qué es un cursor?

¿Qué es una función almacenada?



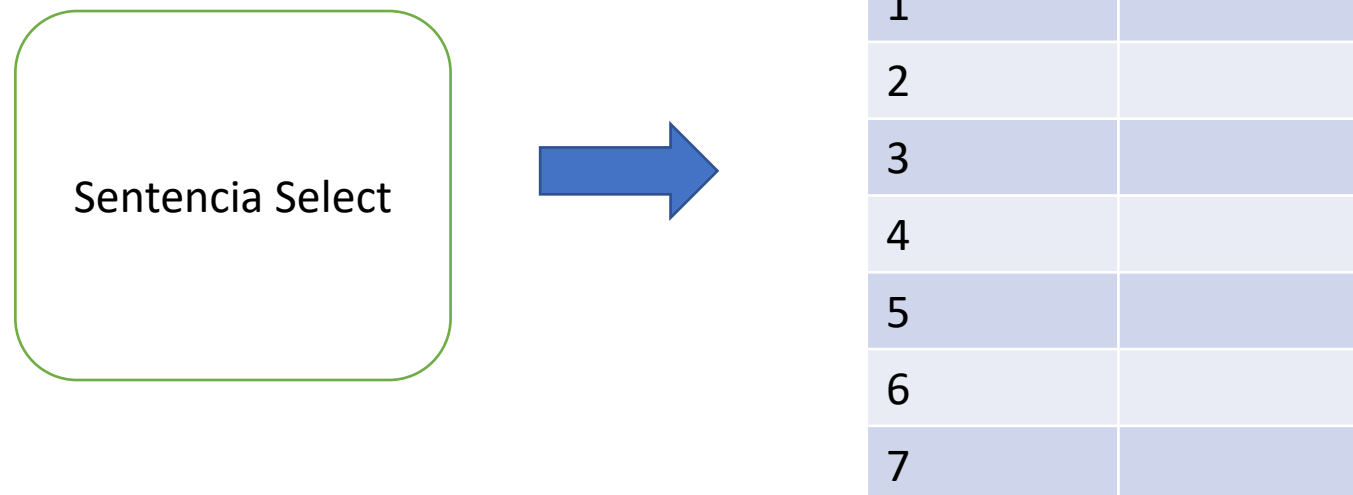
CURSORES

CURSORES SQL SERVER



Un cursor es una variable que nos permite recorrer con un conjunto de resultados obtenidos a través de una sentencia SELECT fila por fila.

- El uso de los cursores es una técnica que permite tratar fila por fila el resultado de una consulta, contrariamente al SELECT SQL que trata a un conjunto de fila.
- Los cursores pueden ser implementador por instrucciones TRANSACT-SQL (cursores ANSI-SQL) o por la API OLE-DB..





¿Cómo se crea un CURSOR?

Cuando trabajemos con cursores debemos seguir los siguientes pasos:

1. Declarar el cursor, utilizando **DECLARE**
2. Abrir el cursor, utilizando **OPEN**
3. Leer los datos del cursor, utilice **FETCH ...INTO**
4. Cerrar el cursor, utilizando **CLOSE**
5. Liberar el cursor, utilizando **DEALLOCATE**

CURSORES SQL SERVER



-- Declaración del cursor

```
DECLARE <nombre_cursor> [INSENSITIVE][SCROLL] CURSOR  
FOR <sentencia_sql>  
FOR [READ ONLY | UPDATE[ OF lista de columnas]]
```

Declarar un cursor

-- Abrir cursor

```
OPEN [ GLOBAL ] <nombre_cursor>
```

Hacer operativo el cursor

-- Leer una fila del cursor

```
FETCH [NEXT | PRIOR | LAST | ABSOLUTE n | RELATIVE n ]  
[FROM][ GLOBAL ] <nombre del cursor> [INTO lista de  
variables ]  
NEXT
```

Permite extraer una fila del cursor

-- Cerrar cursor

```
CLOSE <nombre_cursor>
```

Cierra el cursor y libera la memoria

-- Liberar cursor

```
DEALLOCATE <nombre_cursor>
```

Comando de limpieza, libera el cursor

CURSORES SQL SERVER



- Creamos un cursor del tipo scroll, para mostrar algunos registros puntuales:

```
--Declaramos el cursor
DECLARE c_producto CURSOR SCROLL FOR
SELECT * FROM Compras.productos
--Abrimos el cursor
OPEN c_producto
--recorremos el cursor
FETCH FIRST FROM c_producto
FETCH ABSOLUTE 6 FROM c_producto
FETCH LAST FROM c_producto
FETCH ABSOLUTE 3 FROM c_producto
--cerramos el cursor
CLOSE c_producto
--liberamos el cursor
DEALLOCATE c_producto
```

```
--Declaramos el cursor
DECLARE c_producto CURSOR SCROLL FOR
    SELECT * FROM Compras.productos
--Abrimos el cursor
OPEN c_producto
--recorremos el cursor
FETCH FIRST FROM c_producto
FETCH ABSOLUTE 6 FROM c_producto
FETCH LAST FROM c_producto
FETCH ABSOLUTE 3 FROM c_producto
--cerramos el cursor
CLOSE c_producto
--liberamos el cursor
DEALLOCATE c_producto
```

100 %

Results Messages

	IdProducto	NomProducto	IdProveedor	IdCategoria	CantxUnidad	Pr
1	1	Te Dharamsala	1	1	10 cajas x 20 bolsas	18
	IdProducto	NomProducto	IdProveedor	IdCategoria	Car	
1	6	Mermelada de grosellas de la abuela	3	2		12
	IdProducto	NomProducto	IdProveedor	IdCategoria	CantxUnidad	PrecioUnid
1	110	Aspirina 1000	1	3	Uni	3
	IdProducto	NomProducto	IdProveedor	IdCategoria	CantxUnidad	Preci
1	3	Sirope de regaliz	1	2	12 - bot. 550 ml	10



Cuando recorremos el cursor la variable global **@@FETCH_STATUS** nos indica el estado de la última instrucción FETCH emitida, los valores posibles son:

CURSOR	
1	
2	
3	
4	
5	
6	
7	

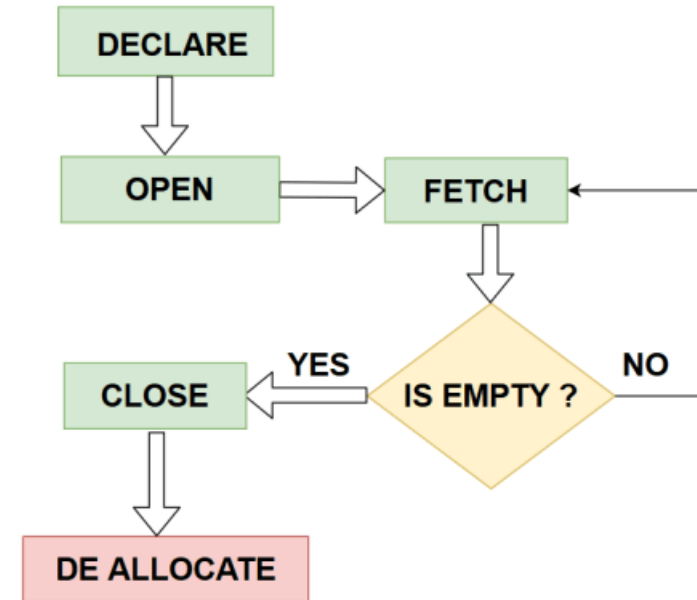
@@FETCH_STATUS

Valor devuelto	Descripción
0	La instrucción FETCH se ejecutó correctamente
-1	La instrucción FETCH no se ejecutó correctamente o la fila estaba más allá del conjunto de resultados.
-2	Falta la fila recuperada.

CURSORES SQL SERVER



```
-- Declaración del cursor
DECLARE <nombreCursor> CURSOR
FOR <sentenciaSQL>
-- Apertura del cursor
OPEN <nombreCursor>
-- Lectura de la primera fila del cursor
FETCH <nombreCursor> INTO <Lista variables>
WHILE (@@FETCH_STATUS = 0)
BEGIN
-- Lectura de la siguiente fila de un cursor
FETCH <nombreCursor> INTO <Lista variables>
END
-- Fin del bucle WHILE
-- Cierra el cursor
CLOSE <nombreCursor>
-- Libera los recursos del cursor
DEALLOCATE <nombreCursor>
```



CURSORES SQL SERVER



- Creamos un cursor, para mostrar el reporte de clientes, considerando el nombre y su dirección, los datos se deben mostrar el consola con PRINT:

```
Declare C_Cliente Cursor For Select NomCliente, DirCliente
from Ventas.clientes
Declare @vc_NomCli Varchar(50), @vc_DirCli varchar(50)
Open C_Cliente
Fetch C_Cliente Into @vc_Nomcli, @vc_Dircli
Print Space(25)+'REPORTE DE CLIENTES'
Print Space(25)+Replicate('*',25)
While @@FETCH_STATUS=0
Begin
Print 'Nombre del Cliente    : '+@vc_NomCli
Print 'Dirección Cliente    : '+@vc_DirCli
Print ''
Fetch C_Cliente Into @vc_Nomcli, @vc_Dircli
End
Print Replicate('*',50)
Close C_Cliente
Deallocate C_Cliente
go
```

Messages	
REPORTE DE CLIENTES *****	
Nombre del Cliente	: Alfreds Futterkiste
Dirección Cliente	: Obere Str. 57
Nombre del Cliente	: Ana Trujillo Emparedados y helados
Dirección Cliente	: Avda. de la Constitucion 2222
Nombre del Cliente	: Antonio Moreno Taqueria
Dirección Cliente	: Mataderos 2312
Nombre del Cliente	: Around the Horn
Dirección Cliente	: 120 Hanover Sq.
Nombre del Cliente	: Berglunds snabbköp
Dirección Cliente	: Berguvsvägen 8
Nombre del Cliente	: Blauer See Delikatessen
Dirección Cliente	: Forsterstr. 57
Nombre del Cliente	: Blondel père et fils
Dirección Cliente	: 24, place Kleber Estrasburgo

CURSORES ANIDADOS



- Muestre el nombre y dirección del cliente, por cada uno mostrar Id de Pedido y fecha de pedido generados., los datos se deben mostrar el consola con PRINT:

```
Declare C_Cliente Cursor For Select IdCliente, NomCliente, DirCliente from Ventas.clientes
Declare @vc_Idcli Varchar(5), @vc_NomCli Varchar(50), @vc_DirCli varchar(50)
Open C_Cliente
Fetch C_Cliente Into @vc_Idcli, @vc_Nomcli, @vc_Dircli
Print Space(25)+'REPORTE DE CLIENTES'
Print Space(25)+Replicate('*',25)
While @@FETCH_STATUS=0
Begin
Print 'Nombre del Cliente : '+@vc_NomCli
Print 'Dirección Cliente : '+@vc_DirCli
Print ''
Declare C_PedidoCli Cursor For Select IdPedido, FechaPedido From Ventas.pedidoscabe
Where IdCliente=@vc_Idcli
Declare @vc_IdPed int, @vc_FecPed Date
Open C_PedidoCli
Fetch C_PedidoCli Into @vc_IdPed, @vc_FecPed
While @@FETCH_STATUS =0
Begin
Print SPACE(20) + LTRIM(@vc_IdPed) + SPACE(20) + CONVERT(VARCHAR(16),@vc_FecPed,6)
Print SPACE(20) + '-----'
Fetch C_PedidoCli Into @vc_IdPed, @vc_FecPed
End
Close C_PedidoCli
Deallocate C_PedidoCli
Fetch C_Cliente Into @vc_Idcli, @vc_Nomcli, @vc_Dircli
End
Print Replicate('*',50)
Close C_Cliente
Deallocate C_Cliente
go
```

Messages	
REPORTE DE CLIENTES *****	
Nombre del Cliente	: Alfreds Futterkiste
Dirección Cliente	: Obere Str. 57
10643	25 Aug 97
10692	03 Oct 97
10702	13 Oct 97
10835	15 Jan 98
10952	16 Mar 98
11011	09 Apr 98
Nombre del Cliente	: Ana Trujillo Emparedados y helados
Dirección Cliente	: Avda. de la Constitucion 2222
10308	18 Sep 96
10625	08 Aug 97
10759	28 Nov 97
10926	04 Mar 98

CURSORES CASO INTEGRADOR



La empresa TUCOMBRANZA.COM requiere un procedimiento almacenado para poder tener una tabla con la información de clientes morosos, para ello se debe implementar un cursor que permita insertar el código, razón social de cliente, monto de la deuda y mora de los clientes con factura pendiente de pago (estado de factura 2) en la tabla que debe ser previamente creada, considere los siguientes criterios:

Si es menos a 200 soles será del 10% de la deuda.

Si es mayor igual a 200 y menos a 1000, será del 15% de la deuda.

Si es mayor o igual a 1000 será del 20% de la deuda.

Pasos:

Use la BD VENTASUPN

Crear la tabla TB_MOROSOS

Crear el procedimiento que incluya el cursor y proceso solicitado

Mostar los resultados



Consideraciones finales sobre el manejo de un cursor

Cuando trabajamos con cursores debemos considerar:

- Cuando un cursor está cerrado, no se puede leer.
- Cuando leemos un cursor debemos comprobar el resultado de la lectura utilizando los atributos de los cursores.
- Cuando se cierra el cursor, es ilegal tratar de usarlo.
- Es ilegal tratar de cerrar un cursor que ya está cerrado o no ha sido abierto.



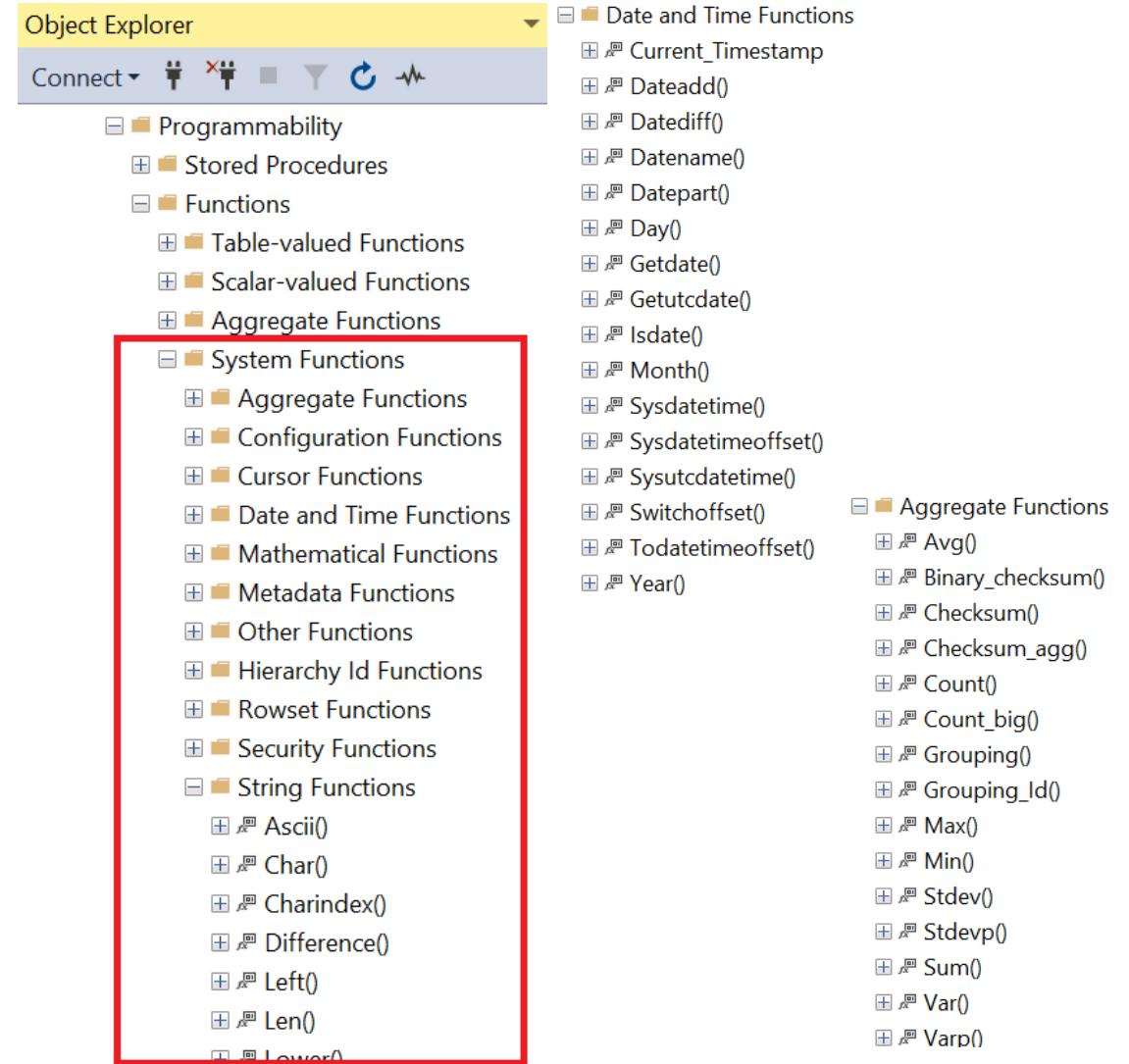
FUNCIONES ALMACENADAS

FUNCIONES ALMACENADAS



Definición

- Las **funciones** se utilizan normalmente para calcular un valor. Existen muchas funciones PREDEFINIDAS como **GETDATE()**, **SUM()**, **AVG()** y **DAY()**, **MONTH()**, **DATEIFF()**. Los desarrolladores también crean sus propias funciones al escribir aplicaciones.
- El código para una función en SQLServer debe contener una sentencia **RETURN**.



FUNCIONES ALMACENADAS



Las funciones almacenadas se crean y se almacenan en la base de datos como un objeto. Al igual que los procedimientos almacenados, las funciones almacenadas pueden aceptar parámetros y devolver valores, lo que hace que sean muy útiles para la lógica de negocios complejos que se puede utilizar repetidamente.

Sintaxis para creación

```
CREATE FUNCTION nombre_funcion (@parametro tipo_dato)
RETURNS tipo_dato_retorno
AS
BEGIN
    -- Código de la función
    RETURN valor_retorno
END
```

FUNCIONES ALMACENADAS



Ejemplo 1 - Creación/Reemplazo de función almacenada

```
USE Negocios
GO

CREATE FUNCTION SUMAR_NUMEROS(@P_NUM1 SMALLINT, @P_NUM2 SMALLINT)
    RETURNS smallint
AS
BEGIN
    DECLARE @p_resultado SMALLINT
    SET @p_resultado = @P_NUM1 + @P_NUM2
    RETURN @p_resultado
END
```

Ejemplo 2 - Ejecución de función almacenada

```
SELECT dbo.SUMAR_NUMEROS(35, 27)
-- 0
PRINT dbo.SUMAR_NUMEROS(35, 27)
```

A screenshot of the SQL Server Enterprise Manager interface. At the top, there is a zoom level of 83 % and a back arrow. Below this are two tabs: 'Results' (active) and 'Messages'. The 'Results' tab displays a single row of data. The first column is labeled '(No column name)' and contains the value 62.

(No column name)
62

FUNCIONES ALMACENADAS



Ejemplo 4

Usamos la función **dbo.SUMAR_NUMEROS**, para realizar una suma con los campos **unidadesEnExistencia** y **unidadesEnPedido** de la tabla **Compras.productos**

```
SELECT NomProducto, UnidadesEnExistencia, UnidadesEnPedido,  
dbo.SUMAR_NUMEROS(UnidadesEnExistencia, UnidadesEnPedido) Suma  
FROM Compras.productos
```

Results Messages				
	NomProducto	UnidadesEnExistencia	UnidadesEnPedido	Suma
1	Te Dharamsala	39	0	39
2	Cerveza tibetana Barley	17	40	57
3	Sirope de regaliz	13	70	83
4	Especias Cajun del chef Anton	53	0	53
5	Mezcla Gumbo del chef Anton	0	0	0
6	Mermelada de grosellas de la abuela	120	0	120
7	Peras secas organicas del tio Bob	15	0	15
8	Salsa de arandanos Northwoods	6	0	6
9	Buey Mishi Kobe	29	0	29
10	Pez espada	31	0	31
11	Queso Cabrales	22	30	52
12	Queso Manchego La Pastora	86	0	86



Ejemplo 5

Operaciones

Crear una función que realiza una operación entre 2 números según el tipo de operación indicada en el parámetro **P_OPERACION**:

- + -> SUMA
- - -> RESTA
- * -> MULTIPLICACIÓN
- / -> DIVISIÓN
- Otro -> NULL

FUNCIONES ALMACENADAS



Ejemplo 5

```
CREATE FUNCTION OPERAR_NUMEROS (  
@P_NUM1 NUMERIC(5,2),  
@P_NUM2 NUMERIC(5,2),  
@P_OP CHAR(1)  
) RETURNS NUMERIC(8,2)  
AS  
BEGIN  
declare @V_RES numeric(8,2)  
SET @V_RES=CASE  
    WHEN @P_OP = '+' THEN  
        @P_NUM1 + @P_NUM2  
    WHEN @P_OP = '-' THEN  
        @P_NUM1 - @P_NUM2  
    WHEN @P_OP = '*' THEN  
        @P_NUM1 * @P_NUM2  
    WHEN @P_OP = '/' THEN  
        @P_NUM1 / @P_NUM2  
    END  
RETURN @V_RES  
END  
go
```

FUNCIONES ALMACENADAS



Invocando a la función desde un bloque T SQL

```
DECLARE
@V_NUMA NUMERIC, @V_NUMB NUMERIC, @V_OPERACION CHAR(1), @V_RESULTADO NUMERIC(8,2)
BEGIN
SET @V_NUMA = 10
SET @V_NUMB = 30
SET @V_OPERACION = '+'
SET @V_RESULTADO = dbo.OPERAR_NUMEROS(@V_NUMA,@V_NUMB,@V_OPERACION)
PRINT TRIM(STR(@V_NUMA)) + ' ' + @V_OPERACION + ' ' + TRIM(STR(@V_NUMB)) + '=' + TRIM(STR(@V_RESULTADO))

SET @V_OPERACION = '*'
SET @V_RESULTADO = dbo.OPERAR_NUMEROS(@V_NUMA,@V_NUMB,@V_OPERACION)
PRINT STR(@V_NUMA) + ' ' + @V_OPERACION + ' ' + STR(@V_NUMB) + '=' + STR(@V_RESULTADO)

SET @V_RESULTADO = dbo.OPERAR_NUMEROS(23,56,'*')
PRINT '23 * 56' + ' = ' + STR(@V_RESULTADO)
END
```

FUNCIONES ALMACENADAS



Ejemplo 6

Crear una función que retorne la fecha actual del sistema, tendrá como parámetro de entrada el formato de salida presentación de la fecha.

```
CREATE FUNCTION OBTENER_FECHA (  
@P_FORMATO char(1) -- d= fecha corta, D= fecha texto  
) RETURNS VARCHAR(50)  
AS  
BEGIN  
    declare @V_FECHA VARCHAR(50)  
    SELECT @V_FECHA = FORMAT(getdate()),@P_FORMATO,'es-es')  
  
    RETURN @V_FECHA  
END;
```

Invocando a la función OBTENER_FECHA:

```
-- Verificando el stored function:  
select dbo.obtener_fecha('D') as fecha
```

FUNCIONES ALMACENADAS



Sintaxis para eliminar funciones

```
DROP FUNCTION name_function;
```

Ejemplo 7

```
DROP FUNCTION OBTENER_FECHA;
```



Ejemplo 8

Crear una función en el esquema compras que reciba como parámetro el ID de un proveedor y que devuelva el **promedio de precios** de los productos que abastece dicho proveedor.

Luego genere una consulta que muestre la lista de proveedores y el promedio de precios de los productos que abastece.

FUNCIONES ALMACENADAS



```
CREATE FUNCTION Compras.fu_promedio_precio(@P_IDPROVEEDOR INT)
RETURNS DECIMAL(12,2)
AS
BEGIN
DECLARE @V_PROMEDIO_PRECIO DECIMAL(12,2)
SELECT @V_PROMEDIO_PRECIO = AVG(PrecioUnidad) FROM
Compras.productos
WHERE IdProveedor = @P_IDPROVEEDOR

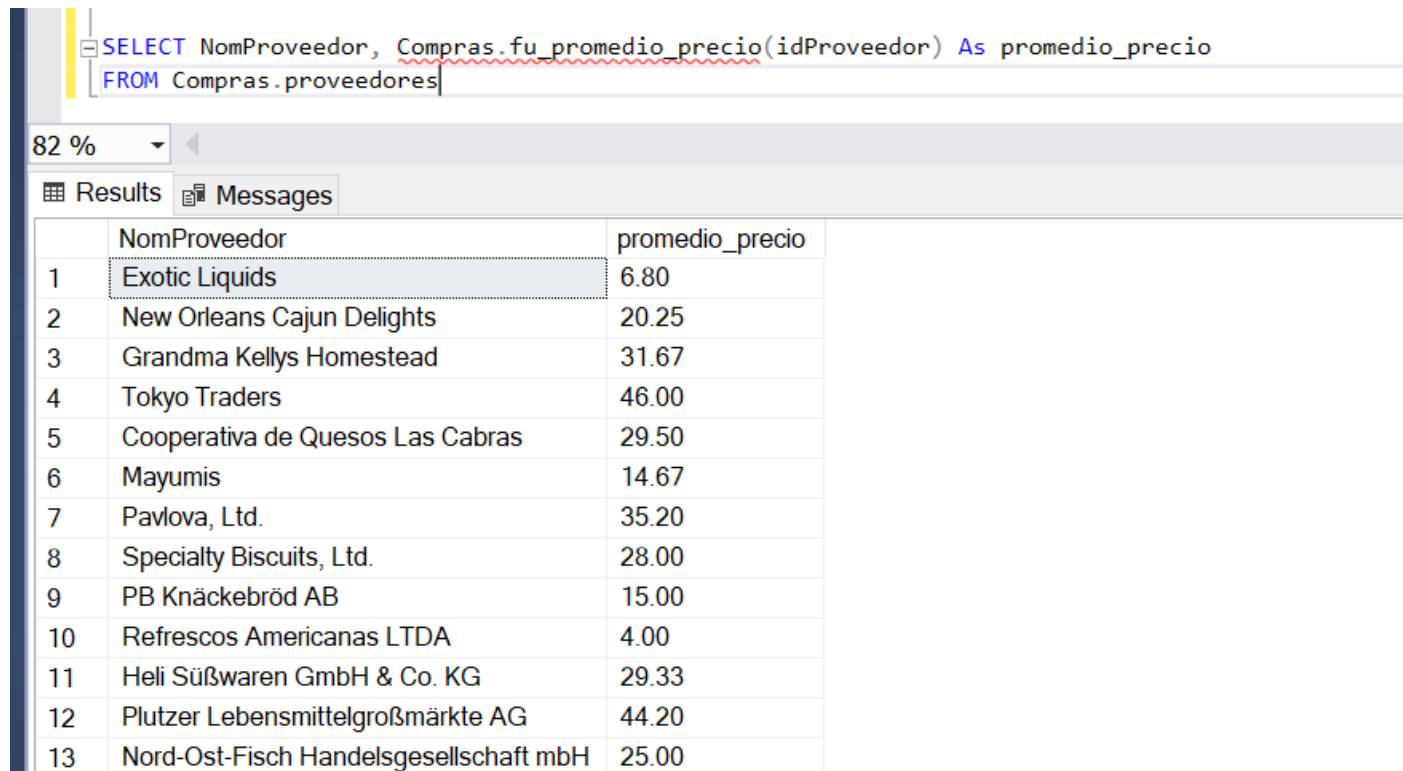
RETURN @V_PROMEDIO_PRECIO
END
```

FUNCIONES ALMACENADAS



--PROBAR LA FUNCIÓN

```
SELECT NomProveedor, Compras.fu_promedio_precio(idProveedor) As promedio_precio
FROM Compras.proveedores
```



The screenshot shows a database query editor with the following SQL query:

```
SELECT NomProveedor, Compras.fu_promedio_precio(idProveedor) As promedio_precio
FROM Compras.proveedores
```

Below the query editor, there is a results pane with a zoom level of 82%. The results are displayed in a table with two columns: NomProveedor and promedio_precio. The table contains 13 rows of data.

	NomProveedor	promedio_precio
1	Exotic Liquids	6.80
2	New Orleans Cajun Delights	20.25
3	Grandma Kellys Homestead	31.67
4	Tokyo Traders	46.00
5	Cooperativa de Quesos Las Cabras	29.50
6	Mayumis	14.67
7	Pavlova, Ltd.	35.20
8	Specialty Biscuits, Ltd.	28.00
9	PB Knäckebröd AB	15.00
10	Refrescos Americanas LTDA	4.00
11	Heli Süßwaren GmbH & Co. KG	29.33
12	Plutzer Lebensmittelgroßmärkte AG	44.20
13	Nord-Ost-Fisch Handelsgesellschaft mbH	25.00



¿Preguntas o comentarios?



CONCLUSIONES

Un cursor es un conjunto de registros devuelto por una instrucción SQL. Nos permitirá procesar un conjunto de registros obtenidos con la ejecución de una sentencia SELECT.

Los cursores pueden ser procesados por el bucle WHILE, para recorrer cada registro.

Una función almacenada es un bloque que devuelve un valor. Al igual que los procedimientos, las funciones se guardan en la base de datos como objetos de un esquema específico.

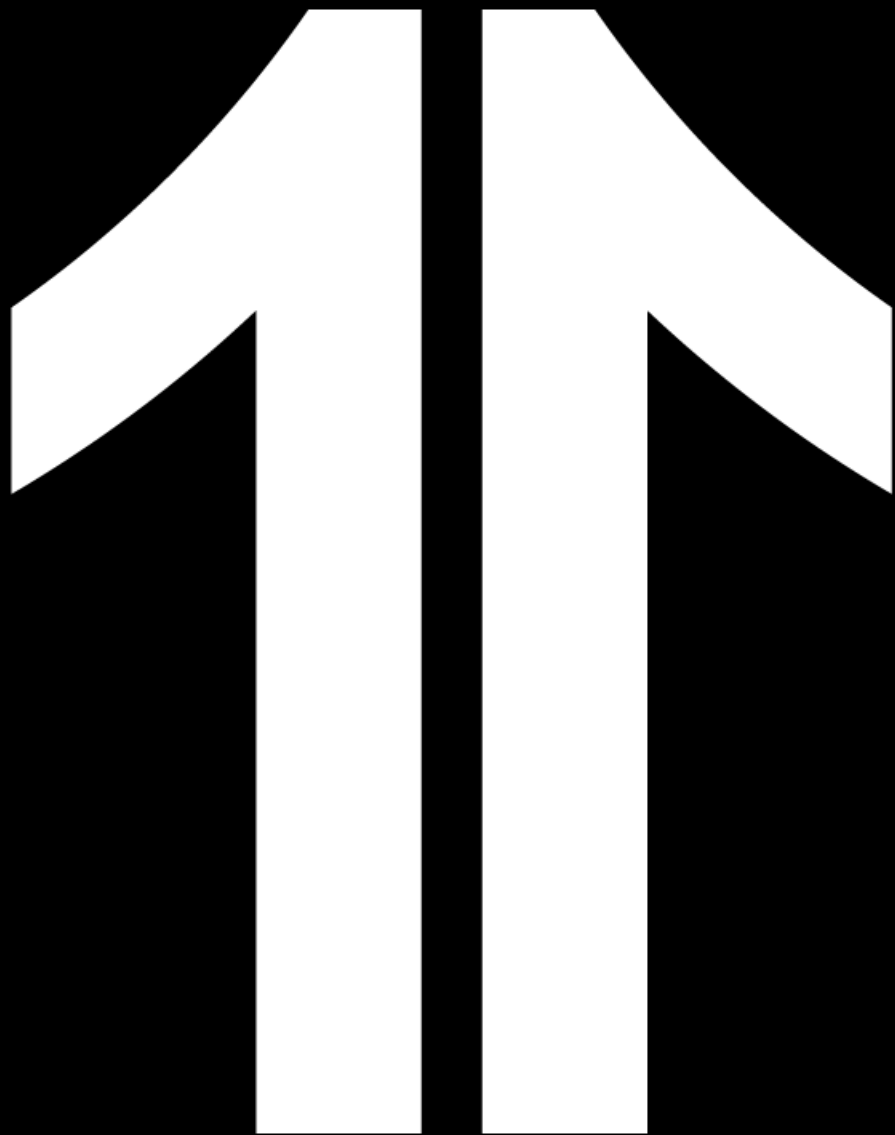


REFERENCIAS BIBLIOGRÁFICAS



Documentación de Microsoft SQL

- Cursores en SQL Server(<https://learn.microsoft.com/es-es/sql/relational-databases/cursors?view=sql-server-ver16>)
- DECLARE CURSOR (Transact-SQL) (<https://learn.microsoft.com/es-es/sql/t-sql/language-elements/declare-cursor-transact-sql?view=sql-server-ver16>)
- Funciones predefinidas (<https://learn.microsoft.com/es-es/sql/t-sql/functions/functions?view=sql-server-ver16>)
- CREATE FUNCTION (Transact-SQL) (<https://learn.microsoft.com/es-es/sql/t-sql/statements/create-function-transact-sql?view=sql-server-ver16>)



GRACIAS

